

PyGeek 2026 구두발표 대본 —

윤준하

대상 파일: docs/reference/PyGeek2026_발표_윤준하_rev10.pptx (18장 · 영문 슬라이드 · 발표자 노트 포함)

일시: 2026-08-20(목) 14:10-14:30 · Oral

Presentation #1 · 홍익대 와우관 2층

슬롯 20분 = **발표 15분 + Q&A 4분 + 교체 1분** · 발표 언어: 한국어

이 대본은 외워서 말하는 전제다. 문장을 그대로 읽지 않는다. 막히면 각 장의 ► 큐 한 줄로 뼈대를 복원한다.

시간 배분표

장	쪽	슬라이드	구간	배정
1	—	Title	0:00-0:25	25초
2	1	Contents	0:25-0:45	20초
3	2	Introduction — 실험 무대(DILAB)	0:45-1:30	45초
4	3	Introduction — 벡터 레이어	1:30-2:10	40초
5	4	Introduction — 두 가지가 함께 바뀐다 ★	2:10-3:25	75초
6	5	Introduction — 원인을 잘못 짚으면	3:25-4:05	40초
7	6	Proposed Method — 중간 조건 ★★	4:05-5:35	90초
8	7	Proposed Method — 시스템 구조	5:35-6:15	40초
9	8	Performance Evaluation — 실험 설계	6:15-7:05	50초
10	9	Experiment Result — 지표 읽는 법	7:05-7:55	50초
11	10	Experiment Result — 저장소 ★	7:55-9:00	65초
12	11	Experiment Result — 위치 ★	9:00-10:10	70초
13	12	Experiment Result — 지표 해설 ②	10:10-10:40	30초
14	13	Experiment Result — 시간·효과 크기	10:40-11:30	50초
15	14	Experiment Result — 등가성	11:30-12:30	60초
16	15	Conclusion ★	12:30-13:55	85초
17	16	References	13:55-14:05	10초
18	17	Thank You	14:05-14:20	15초

중간 점검: 슬라이드 9(실험 설계)를 넘길 때 **7분 5초**면 정상. 8분이 넘었으면 아래 <생략 가능>을 버린다.

배정에는 여유가 들어 있다. 외운 것을 말하는 속도로 약 13분이고, 나머지는 호흡·그림 짚기·슬라이드 넘김 몫이다. 시계가 배정보다 앞서 있어도 서두를 필요가 없다. 여유가 가장 많은 곳은 **7번(중간 조건)과 12번(위치 요인)** 이고, 이 두 장은 천천히 가라고 일부러 눌러 둔 것이다.

1 Title

0:00-0:25

▶ 이름 → 제목 → 한 줄 요지

안녕하십니까. 성결대학교 윤준하입니다.

발표드릴 논문의 제목은 「벡터 데이터베이스 이전에서, 저장소 교체와 임베딩 위치 효과를 분리하는 요인 분해 방법」입니다.

제목을 한 문장으로 풀면 이렇습니다. **데이터베이스를 옮겼더니 검색 품질이 달라졌을 때, 무엇 때문에 달라졌는지를 가려내는 방법**입니다.

▶ **배경** → **문제** → **방법** → **실험·결과** → **결론**

발표 순서입니다.

이 문제가 나온 **서비스 배경**을 먼저 보여 드리고, **문제**가 무엇인지, 저희가 제안하는 **방법**이 무엇인지 말씀드리겠습니다. 이어서 **실험과 결과**를 보여 드리고, **결론**으로 마무리하겠습니다.

▶ **딜랩 소개 → 왼쪽 구조 → 오른쪽 화면 → 왜 이
전이 필요했나**

연구가 시작된 자리부터 보여 드리겠습니다.

화면은 **딜랩이라는 제품 리뷰 분석 데모 서비스**입니다. 소비자 리뷰와 전문가 리뷰를 벡터 데이터베이스에 넣어 두고, 질문이 오면 관련 리뷰를 검색해 **근거를 인용한 제품 평가 리포트**를 만들어 줍니다.

(왼쪽 그림을 짚으며) 왼쪽이 시스템 구조입니다. 리뷰가 임베딩을 거쳐 벡터 저장소에 쌓이고, 질문이 오면 저장소를 검색해 답을 만듭니다.

(오른쪽 그림을 짚으며) 오른쪽이 실제 화면입니다. 다섯 개 축으로 제품을 채점하고, 판단마다 원문 근거가 붙습니다.

그런데 딜랩 데모는 **pgvector라는 저장소 위에** 만들어져 있고, **실제 운영을 맡을 환경은 오라클 데이터베이스로 표준화**되어 있습니다. 실운영으로 가려

면 벡터 레이어를 오라클로 옮겨야 했습니다. 오늘 발표할 질문이 바로 그 이전 과정에서 나왔습니다.

4

Introduction — 벡터 레이어

1:30-2:10

▶ RAG 세 부분 → 벡터 레이어가 교체 1순위 →
흔한 경로 = 딜랩의 경로

방금 본 딜랩 같은 검색 증강 시스템은 크게 세 부분
입니다. 글을 숫자 목록으로 바꾸는 임베딩 모델, 그
숫자를 쌓아 두고 비슷한 것을 찾아 주는 저장소, 그
리고 찾아온 근거로 답을 쓰는 언어 모델입니다.

(그림 위쪽을 짚으며) 앞의 두 부분, 벡터를 다루는
부분이 서비스를 띄운 뒤 가장 자주 갈아 끼우게 되
는 자리입니다.

(그림 아래쪽을 짚으며) 가장 흔한 교체 경로가 화면
아래에 있습니다. 일반 데이터베이스에 벡터 기능을
없어 쓰다가, 임베딩 계산까지 데이터베이스가 직접
해 주는 제품으로 옮겨 가는 경우입니다. 딜랩의 이
전 이 바로 이 경로였습니다.

▶ 바뀌는 게 둘 → ①저장소 ②위치 → 왜 같이 움직이나 → 원인 분리 불가

문제는 이 이전에서 시작됩니다. 한 번 옮길 때 **바뀌는 것이 하나가 아니라 둘입니다.**

(두 상자를 짚으며) 왼쪽이 pgvector를 얹은 지금 시스템, 오른쪽이 오라클로 옮긴 시스템입니다. 오늘 말씀드릴 내용은 제품의 우열이 아니라, **어느 이전이나 나타나는 구조**입니다.

첫째, **저장소**가 바뀝니다. 검색 엔진이 바뀌고 찾는 방식도 바뀝니다. 기존 저장소는 빠르게 찾는 대신 가끔 놓치는 방식이고, 새 저장소는 전부 훑는 방식입니다.

둘째, **임베딩을 계산하는 위치**가 바뀝니다. 데이터베이스 바깥에서 하던 계산을 안에서 하게 됩니다.

(붉은 두 줄을 짚으며) 붉게 표시된 두 줄이 실제로 달라지는 항목입니다.

저장소와 위치는 왜 같이 움직일까요. **데이터를 경계 안에 두려고** 이전을 결정하는 경우가 많고, 그러면 질문이 밖으로 나가는 것도 막고 싶어져 **임베딩이 데이터베이스 안으로 따라 들어옵니다.** 그런데 데이터베이스 안에 넣을 수 있는 모델은 **크기 제한**이 있어서, 대체로 더 작은 모델로 바꿉니다.

결국 이전 전과 후만 비교해서는, **달라진 원인이 저장소인지 모델인지** 짚을 수 없습니다.

▶ 하락해도 모름 / 그대로여도 모름 → 처방이 어긋남

원인을 못 짚으면 무슨 일이 생길까요.

(왼쪽 상자) **품질이 떨어진 경우**, 저장소 탓인지 모델 탓인지 알 수 없습니다.

(오른쪽 상자) **변화가 없는 경우도** 마찬가지입니다. 정말 영향이 없었는지, 한쪽이 올리고 한쪽이 내려 상쇄됐는지 구분되지 않습니다.

어느 쪽이든 결론은 "**모르겠다**" 가 됩니다. 그러면 처방이 어긋납니다. 저장소를 범인으로 지목해 되돌려 놓아도 **품질은 돌아오지 않습니다**. 실제 원인이 임베딩 모델 쪽에 있었다면, 저장소를 되돌리는 일은 헛수고이기 때문입니다.

이 발표의 핵심입니다. 호흡을 두고, 그림을 짚으면서 천천히 갑니다.

▶ 이사 비유(집=저장소·가구=벡터) → B 만드는 법 → $A \rightarrow B$ 저장소만 / $B \rightarrow C$ 위치만 → 재임베딩 불필요

저희 제안은 두 조건 사이에 **중간 조건 하나**를 끼워 넣는 것입니다. 이사에 비유해 보겠습니다.

집을 옮기면서 가구까지 한꺼번에 새로 사면, 새집 분위기가 달라졌을 때 **집 때문인지 가구 때문인지 알 수 없습니다**. 알아내는 방법은 하나뿐입니다. **먼저 쓰던 가구를 그대로 새집에 옮겨 보고, 그다음에 가구를 바꾸는 것**입니다.

이 비유에서 **집이 저장소**이고, **가구가 벡터**입니다.

(가운데 상자를 짚으며) 화면 가운데 조건 B가 "쓰던 가구를 그대로 옮긴 새집"입니다. **지금 시스템이 이미 만들어 둔 벡터를, 다시 계산하지 않고 새 저장소에 그대로 실어 올립니다**. 그래서 조건 B는 **저장소만**

새것이고, 임베딩 모델과 계산 위치는 원래 그대로입니다.

(왼쪽 화살표) **A에서 B로 갈 때는 저장소만 달라집니다.** (오른쪽 화살표) **B에서 C로 갈 때는 임베딩 위치만 달라집니다.** 한 번에 하나씩만 보게 됩니다.

(화살표 위 숫자를 가리키며) 화살표 위의 숫자 두 개가 그 두 구간의 결과인데, 잠시 뒤 자세히 말씀드리겠습니다.

실무에서 좋은 점이 하나 있습니다. **벡터를 다시 만들지 않으니 임베딩 비용이 들지 않고, 운영 중인 서비스를 멈추지 않아도 됩니다.** 이전을 결정하기 전에 미리 해 볼 수 있다는 점이 이 방법의 쓸모입니다.

▶ 왼쪽 두 번 왕복 → 오른쪽 안에서 끝 → 설정 고정

세 조건이 실제 시스템에서 어떤 모습인지 구조로 보여 드리겠습니다.

(왼쪽) 옮기기 전 시스템은 애플리케이션이 **바깥의 임베딩 서비스를 부르고**, 받은 벡터를 다시 저장소로 보냅니다. **네트워크 밖으로 두 번 나갔다 옵니다.**

(오른쪽) 옮긴 뒤에는 임베딩과 검색이 **모두 데이터 베이스 안에서** 끝납니다. 밖으로 나가는 일은 한 번으로 줄고, 외부 모델 호출은 없어집니다.

꼭 말씀드릴 것이 하나 있습니다. **검색 규칙과 설정 값은 세 조건에서 완전히 똑같이 고정했습니다.** 비교하려는 요인 말고 다른 것이 섞이면 안 되기 때문입니다.

이 장을 넘길 때 7분 5초면 정상 속도입니다.

▶ 1,347 청크 · 43 질문 → 출처 13+30 → 지표 셋 → 표의 색

실험 설정입니다.

대상은 앞서 소개드린 딜랩의 리뷰 청크 1,347개이고, 미리 정해 둔 질문 43개를 세 조건에 똑같이 던졌습니다.

질문의 출처를 분명히 말씀드리면, 13개는 실제 서비스 로그에서 뽑았고, 30개는 평가 항목과 구매 단계를 고르게 덮도록 저희가 만들었습니다. 두 묶음이 다르게 행동하지 않는지는 따로 확인했습니다.

지표는 세 가지입니다. 검색 결과가 얼마나 같은지, 찾아온 근거가 질문에 얼마나 맞는지를 0점에서 2점으로 매긴 값, 그리고 검색에 걸린 시간입니다.

(표를 짚으며) 파란 칸이 저장소 요인으로 바뀌는 항목, 붉은 칸이 위치 요인으로 바뀌는 항목입니다. 색

이 없는 칸은 세 조건에서 그대로입니다.

〈생략 가능〉 측정은 가장 낮은 등급 인스턴스에서 했습니다.

숫자를 처음 다루는 자리입니다. 서두르지 않습니다.

▶ 겹침 0~1 → p값: 질문 다시 뽑기 반복 → 자주 나오면 우연 / 드물면 진짜

결과를 보여 드리기 전에, 결과를 읽을 **잣대 두 개**를 먼저 설명드리겠습니다.

(왼쪽) 첫째는 **두 검색 결과가 얼마나 겹치는지**입니다. 상위 10개가 완전히 같으면 1, 하나도 안 겹치면 0입니다.

(오른쪽) 둘째가 **p값**입니다. 저희가 쓴 질문 43개는 가능한 질문 가운데 일부를 뽑은 것이라, 다른 43개를 뽑았다면 숫자도 조금 달랐을 것입니다. 그래서 **두 조건이 사실은 똑같다고 가정해 놓고, 질문 뽑기를 다시 반복하면 지금만 한 차이가 얼마나 자주 나오는지를** 봅니다. 그 비율이 p값입니다.

자주 나오는 차이라면 **우연으로 설명됩니다. 좀처럼 안 나오는 차이**라면 **우연이라 보기 어렵습니다.**

(그림을 짚으며) 회색 막대들이 우연이 만들 수 있는
차이의 분포이고, 관측된 값이 분포의 어디에 떨어지
는지로 판정합니다. 화면의 예시 값은 바로 다음에
보실 저장소 구간입니다.

▶ 결론: 변화 미검출 → 겹침 0.971(43개 중 37개 동일) → p 0.523 → 6개 차이 = 1.6% → 시간은 오히려 단축

첫 번째 결과, 저장소만 바꾼 A에서 B 구간입니다.

결론부터 말씀드리면, 품질 변화가 검출되지 않았습니다.

상위 10개의 겹침이 **0.971**입니다. 같은 벡터를 실었더니, 질문 43개 가운데 37개에서 상위 10개가 정확히 같았습니다.

근거 품질의 p 값은 **0.523**입니다. 방금 설명드린 것대로 읽으면, 두 저장소가 똑같다고 해도 절반이 넘는 경우에 나올 만한 차이라는 뜻입니다. 우연으로 충분히 설명됩니다.

나머지 6개 질문에서 차이가 난 이유는, 기존 저장소가 빠르게 찾는 대신 가끔 놓치는 방식이기 때문입니다. 놓친 양은 1.6퍼센트 수준이었습니다.

검색 시간은 **150밀리초에서 110밀리초로 오히려 조금 줄었습니다.** 저장소만 바꾸는 선택에서는 잃은 것이 없었습니다.

▶ 겹침 0.971→0.175 → 품질 1.27→1.01 (p 0.003) → 같은 저장소인데 딴 근거 → 앞뒤만 비교했다면

두 번째 결과, 임베딩 위치만 바꾼 B에서 C 구간입니다. 여기서도 그림이 완전히 달라집니다.

방금 0.971이던 겹침이 **0.175로 떨어졌습니다**. 거의 다른 목록이 돌아왔다는 뜻입니다.

근거 품질도 2점 만점에 **1.27점에서 1.01점으로, 약 20퍼센트 떨어졌습니다**. p 값은 **0.003**입니다. 두 조건이 똑같다면 천 번에 세 번 나올 차이라, 우연으로 보기 어렵습니다.

강조하고 싶은 부분은 이것입니다. **저장소도 같고 검색 방식도 같은데, 같은 질문에 대체로 다른 근거가 돌아왔습니다**. 앞 구간에서 저장소 효과가 검출되지 않았으므로, 관측된 변화는 사실상 전부 임베딩 위치와, 위치에 딸려 온 모델 크기 제한에서 나온 것입니다.

한 박자 쉬고, 청중을 보며 말합니다.

만약 이전 전후만 비교했다면 어땠을까요. 저장소 탭으로 돌리고, 저장소를 되돌리고, 품질은 돌아오지 않았을 것입니다.

13

Experiment Result — 지표 해설 ②

10:10-10:40

▶ 같은 잣대 → 겹침 3/17 → 분포 꼬리 바깥

저장소 구간에 됐던 잣대 두 개를 위치 구간에 그대로 댄 그림입니다.

(왼쪽) 겹침은 0.971에서 0.175로 떨어졌습니다. 두 목록을 합치면 17개인데, 겹치는 것이 3개뿐입니다.

(오른쪽) p값은 0.523에서 0.003으로 내려갔습니다. 관측된 값이, 우연이 만드는 분포의 꼬리 바깥에 나가 있습니다.

같은 잣대를 댔는데 이만큼 다른 그림이 나온다는 것은, 두 요인이 실제로 분리되었다는 뜻입니다.

▶ 시간 150→110→947(8배) → p는 "우연인가",
효과 크기는 "얼마나 큰가" → 0.098 무시 /
0.473 중간

품질 말고 **시간**도 보겠습니다.

(왼쪽 패널) 저장소만 바꾼 구간에서는 검색이 150밀리초에서 110밀리초로 오히려 빨라졌습니다. 그런데 **임베딩을 안으로 들인 구간에서는 110밀리초에서 947밀리초로, 약 여덟 배 느려졌습니다.** 질문이 올 때마다 데이터베이스 안에서 임베딩 모델을 돌려야 하기 때문입니다. 다만 가장 낮은 등급 인스턴스에서 켜 값이라, 장비를 올리면 절대치는 달라질 수 있습니다.

(오른쪽 패널) 품질 차이는 **효과 크기**라는 값으로도 보겠습니다. p값이 "이 차이가 우연인가"를 묻는 값이라면, 효과 크기는 "그 차이가 실제로 얼마나 큰가"를 묻는 값입니다. 표본이 아주 많으면 하찮은 차이도 p값이 작게 나올 수 있어서, 두 값을 같이 봐야 합니다.

저장소 요인은 **0.098로 무시할 수준**이고, 위치 요인은 **0.473으로 통계에서 중간이라 부르는 크기**입니다.

느려진 것도, 품질이 떨어진 것도 모두 위치 요인의 몫이었습니다.

▶ 못 찾았다 $\neq$ 없다 $\rightarrow \pm 0.15$ 띠 $\rightarrow$ 왼쪽 이탈, p 0.052 $\rightarrow$ "나빠졌다 볼 근거 없다"까지만

조심스럽게 말씀드릴 부분이 있습니다.

저장소 요인은 차이가 검출되지 않았다고 했는데, "차이를 못 찾았다"와 "차이가 없다"는 다른 말입니다. 데이터가 부족해서 못 찾았을 수도 있습니다.

"차이가 없다"를 주장하려면 검정을 따로 해야 합니다. 2점 만점에서 플러스마이너스 0.15점 안쪽이면 사실상 같다고 보자는 기준을 정하고, 등가성 검정을 했습니다.

(파란 띠를 짚으며) 열린 띠가 그 기준입니다. 신뢰구간이 띠 안에 통째로 들어와야 하는데, 왼쪽 끝이 조금 나가 있습니다. p 값 0.052로, 기준선 0.05를 아슬아슬하게 넘지 못했습니다.

그래서 결론은 이렇게 보고합니다. "같다는 것이 입증되었다"가 아니라, "차이를 찾지 못했고, 있더라도 이 정도 안쪽이다"입니다.

다만 신뢰구간이 뺀 방향은 새 저장소 점수가 더 높은 쪽입니다. 저장소를 바꿔서 품질이 나빠졌다고 볼 근거는 없습니다.

정리와 판단 기준 사이에 호흡 한 번.

▶ 방법 정리 → 관측 정리 → 거래 문장 → 판단 기준 둘 → 한계

정리하겠습니다.

방법 면에서는, 중간에 조건 하나를 두면 **앞뒤 비교 한 번이 요인별 비교 두 번으로 나뉩니다.**

관측 면에서는, 저장소 교체만으로는 품질 변화가 검출되지 않았고, **관측된 변화는 전부 임베딩 위치에서 나왔습니다.**

드리고 싶은 문장은 이것입니다. **임베딩을 데이터베이스 안으로 들이는 것은 성능을 사는 거래가 아니라, 데이터를 경계 안에 두는 대가로 성능을 내주는 거래입니다.** 이번 실험에서 치른 값은 **근거 품질 20 퍼센트와 검색 시간 여덟 배였습니다.**

한 박자.

실무 판단 기준으로 바꾸면 이렇습니다. **저장해 둔 데이터만 경계 안에 두면 되는 경우라면, 저장소 교체로 충분합니다. 질의 경로까지 안에 두어야 한다면, 옮기기 전에 모델 크기가 충분한지 확인해야 합니다. 중간 조건은 그 확인을 이전 결정 전에 할 수 있게 해 줍니다.**

한계도 분명히 말씀드리겠습니다. **한 도메인의 한국어 데이터 하나에서 얻은 결과이고, 채점에 사람이 참여하지 않았습니**다. 시간 수치는 최저 등급 인스턴스 기준이라 일반화할 수 없습니다. 설계상 **위치와 모델 크기가 붙어 있어, 둘을 떼어 놓는 네 번째 조건**이 다음 과제입니다.

17

References

13:55-14:05

읽지 않습니다. 화면만 띄우고 한 문장으로 넘깁니다.

참고한 문헌입니다.

이상으로 발표를 마치겠습니다. 감사합니다.

〈시간이 남을 때만〉 화면에 메일 주소를 띄워 두었습니다. 발표 후에 물어보실 것이 있으시면 편하게 연락 주십시오.

예상 질문과 답변

답변은 짧게 시작해서 필요하면 덧붙이는 순서로
깁니다. 모르는 것은 모른다고 말하는 편이 낫습니
다. 막히면 세 숫자로 돌아옵니다 — 저장소 0.971
/ 위치 0.175 / 시간 8배.

Q1. 왜 더 큰 임베딩 모델을 쓰지 않았습니까?

데이터베이스 안에서 모델을 돌리려면, 그 데이터베
이스가 감당할 수 있는 크기 안에서 골라야 합니다.
저희가 쓴 384차원 모델은 그 제한 안에서 고를 수
있었던 선택지였습니다.

그리고 이 제약 자체가 저희 논문이 다루는 대상이기
도 합니다. 위치를 안으로 옮기면 모델 크기가 따라
온다는 점이, 바로 실무에서 문제가 되는 지점이라고
보았습니다.

Q2. 질문 43개는 너무 적지 않습니까?

말씀하신 대로 표본이 크지 않습니다. 한계로 인정합니다.

그래서 저희는 값 하나만 제시하지 않고 **신뢰구간과 차이의 크기를 함께 보고했고**, 결론도 그 구간 안에서만 이야기하려고 했습니다. 질문 수를 늘리는 것은 저희도 필요하다고 보고 있습니다.

Q3. 질문 30개를 직접 만들었는데, 결과를 믿을 수 있습니까?

30개를 저희가 만든 것은 사실이고, 그래서 **두 묶음이 다르게 행동하는지 따로 확인했습니다.**

절대 점수는 실제 로그에서 나온 13개 쪽이 유의하게 높았습니다. **저희가 만든 질문이 더 어려웠다는 뜻입니다.** 다만 결론의 근거인 **랭킹 일치도는 두 묶음에서 차이가 없었습니다.** 어느 묶음으로 잘라 봐도 결론은 같습니다.

Q4. 관련성 점수를 사람이 매기지 않은 것은 신뢰할 수 있습니까?

한계로 명시했습니다. 사람 평가자가 참여하지 않았 습니다.

다만 채점 기준을 세 조건에 똑같이 적용했기 때문 에, **조건끼리 비교한다는 목적에서는 방향성이 유지 된다고 보았습니다.** 점수의 절대값 자체를 주장하려 는 것은 아닙니다. 사람 평가를 덧붙이는 것은 다음 과제로 생각하고 있습니다.

Q5. 중간 조건이라는 것이 새로운 방법입니까?

구성 요소 하나하나가 새롭다고 말씀드리기는 어렵 습니다. 벡터를 옮겨 심는 것도, 요인을 하나씩 바꿔 보는 것도 익숙한 방식입니다.

저희가 기여라고 생각하는 부분은, **벡터 데이터베이 스 이전이라는 구체적인 의사결정 상황에서 이것을 하나의 절차로 정리하고, 벡터를 다시 만들지 않고 운영 서비스를 멈추지 않고도 할 수 있다는 것을 실 제 측정으로 보인 점입니다.**

Q6. 그러면 이전을 하지 말라는 뜻입니까?

그런 뜻은 아닙니다.

저장해 둔 데이터를 경계 안에 두는 것이 목적이라면, 저장소만 바꾸는 선택으로 충분하다는 것이 저희 결과입니다. 이 경우에는 품질 변화가 검출되지 않았습니다.

질의 경로까지 안에 두고 싶으시다면, **그때는 모델 크기가 충분한지를 먼저 확인하시라**는 것이 저희가 드리는 말씀입니다.

Q7. 저장소 요인에 검색 방식(인덱스) 차이가 섞이지 않았습니까?

섞여 있었고, 그 크기를 재 보았습니다. 기존 시스템은 빠르게 찾는 대신 가끔 놓치는 방식이고 새 시스템은 전부 훑는 방식인데, 그 차이로 기존 시스템이 놓친 부분이 **약 1.6퍼센트** 수준이었습니다.

이 차이를 포함해도 저장소 요인에서는 유의한 품질 변화가 검출되지 않았습니다.

Q8. 데이터 규모가 커져도 같은 결과가 나오니까?

품질 쪽 결론은 방향이 유지될 것으로 기대하지만, 1,347개에서 얻은 숫자를 더 큰 규모에 그대로 주장하지는 않겠습니다. 속도는 분명히 달라집니다.

저희가 제안하는 것은 숫자가 아니라 **절차**입니다. 규모가 커져도, 중간 조건을 끼워 요인을 나누는 절차는 그대로 쓸 수 있습니다.

Q9. 여덟 배 느려졌다는 것은 in-DB 방식이 원래 느리다는 뜻입니까?

가장 낮은 등급 인스턴스에서 낸 결과라서, 그렇게 일반화할 수는 없습니다. 장비를 올리면 절대치는 달라집니다. 다만 **질문이 올 때마다 임베딩 모델 추론이 검색 경로 안에 들어간다는 구조**는 장비와 무관하게 남습니다.

덧붙이면, A와 B의 시간 차이에는 임베딩 시간이 섞이지 않았습니다. **외부 임베딩은 두 조건 공통이라 한 번만 잰고, A·B 차이는 검색 시간만 반영합니다.**

통계·검증 방법에 대한 질문

도구 이름을 먼저 말하지 않습니다. **"파이썬으로 계산했다"**는 방법에 대한 답이 아닙니다. 어떤 검정

을, 왜 골랐는지부터 말하고, 라이브러리는 물어보실 때만 답합니다.

Q10. 검증은 구체적으로 어떻게 하셨습니까?

같은 질의 43개를 세 조건에 똑같이 던졌기 때문에, 대응표본 t검정을 주 검정으로 썼습니다. 질의마다 난이도가 다른데 대응표본은 그 차이를 상쇄해 줍니다.

덧붙이면, 정규성 가정에 기대지 않아도 결론이 같은지 보려고 월콕슨 부호순위 검정을 함께 돌렸고, 두 검정이 같은 결론을 냈습니다. 신뢰구간은 표본이 크지 않아 부트스트랩 1만 회로 잡았고, 효과 크기는 Cohen's d 계열 지표로 함께 보고했습니다. 난수 seed를 고정해 두어 재현이 됩니다.

한 가지 덧붙이면, 자카드 유사도는 검정이 아니라 집합이 얼마나 겹치는지를 나타내는 요약값입니다. 그래서 검정 대신 부트스트랩 신뢰구간만 붙였습니다.

Q11. 검정을 여러 번 하셨는데 다중비교 보정은 하셨습니까?

적용하지 않았습니다.

다만 핵심 주장 두 개는 p값이 **0.523**과 **0.003**이라, 보정 여부로 결론이 바뀌는 자리가 아닙니다. 나머지 검정은 탐색적 확인 목적이었습니다.

〈추가로 물으실 때만〉 보고한 검정 13건 전부에 가장 보수적인 본페로니를 적용해도 **0.044**로 유의성은 유지됩니다.

이 숫자는 먼저 꺼내지 않습니다. 검정을 16건으로 세면 0.055가 되어 넘어갑니다. 물어보셨을 때 답하는 용도이지, 자랑할 숫자가 아닙니다.

**Q12. 등가성 검정의 ± 0.15 마진은 어떤 근거로 정하
셨습니까?**

**이론적으로 도출한 값이 아니라 저희가 정한 임계값
입니다. 0에서 2점 척도에서 7.5퍼센트에 해당하는
폭입니다.**

다만 한 가지 말씀드리고 싶은 것은, **저희는 이 검정
으로 등가성을 주장하지 않았다는 점입니다.** 오히려
기준을 충족하지 못했다고 보고했습니다. 마진을 유
리하게 골라서 얻은 결론이 아닙니다.

**Q13. p값 0.052면 사실상 같다고 봐도 되는 것 아닙
니까?**

기준선을 넘지 못한 것은 사실이라, 저희는 "같다"고
주장하지 않기로 했습니다.

다만 말씀하신 대로 아주 근소한 차이이고, 신뢰구간
이 뻗은 방향도 새 저장소 쪽이 유리한 방향입니다.
그래서 **"품질이 나빠졌다고 볼 근거는 없다"**까지는
말씀드릴 수 있다고 보았습니다. 질문 수를 늘리면
판정이 달라질 여지가 있다고 생각합니다.

Q14. 관련성 점수를 0에서 2점, 세 단계로만 매긴 이유가 있습니까?

채점자가 사람이 아니라 대형 언어모델이기 때문입니다.

척도가 촘촘할수록 같은 입력에 다른 점수가 나옵니다. **무관, 부분, 직접** 세 단계는 경계를 말로 정의할 수 있어서 재현성이 높습니다. 정보검색 평가에서 오래 쓰여 온 등급이기도 합니다.

그리고 저희에게 필요했던 것은 점수의 절대 수준이 아니라 **조건 사이의 차이**였습니다. 세 조건에 같은 잣대를 댔다는 점이 척도의 해상도보다 중요했습니다.

Q15. 채점 건수가 215건인데 왜 43개로 검정하셨습니까?

같은 질의에서 나온 5개 점수가 서로 독립이 아니기 때문입니다.

215건으로 세면 표본을 부풀리게 됩니다. 그래서 질의별로 평균을 내어 **43개 대응쌍**으로 검정했고, 215건 수준의 수치는 참고용으로만 보고했습니다.

Q16. 차이가 없다고 하실 때, 정확히 무엇의 차이입니까?

0에서 2점으로 매긴 관련성 점수의 평균입니다. 숫자가 세 단계로 만들어집니다.

먼저 검색해 온 근거 하나하나에 **0점, 1점, 2점**이 붙습니다. 질의 하나당 다섯 개를 평균 내면 **그 질의의 점수**가 나옵니다. 그 질의별 점수를 43개 전부 평균 낸 것이 조건별 최종 점수입니다.

기존 조건이 **1.228점**, 새 저장소 조건이 **1.270점**이었습니다. **이 0.042점이 저희가 말씀드리는 차이입니다.**

〈예를 들어 달라고 하실 때만〉 향을 묻는 질의 하나에서는 기존이 1.2점, 새 저장소가 0.8점으로 기존이 이겼습니다. 다른 질의에서는 반대가 됩니다.

Q17. p값이 말하는 우연이란 무엇을 가리킵니까?

저희가 질의 43개를 어떻게 뽑았느냐의 우연입니다.

채점 자체는 무작위가 아닙니다. 채점 모델의 온도를 0으로 두어 같은 입력에는 같은 점수가 나옵니다. 우연이 들어오는 자리는 **표본**입니다. 43개는 가능한 질의 중 일부이고, 다른 43개를 뽑았다면 숫자가 달라 집니다.

그래서 p값은 이렇게 읽습니다. **두 조건이 정말 똑같다고 가정했을 때, 질의를 다시 뽑는 일을 반복하면 이 정도 차이가 얼마나 자주 나오는가**입니다.

질의별 승패를 세어 보면 분명해 집니다.

비교	이긴 질의	동점	평균차	p
저장소 교체	14 대 14	15	0.042	0.523
임베딩 이전	26 대 12	5	0.256	0.003

저장소를 바꾼 쪽은 **43개를 14개씩 나눠 가진 무승부**입니다. 두 조건이 똑같더라도 이 정도 차이는 절반이 넘는 경우에 그냥 나옵니다. **우연으로 충분히 설명되니 저희는 아무 주장도 하지 않습니다.**

임베딩을 옮긴 쪽은 **26 대 12로 한쪽이 이겼습니다.** 두 조건이 똑같다면 이런 결과는 천 번에 세 번 나옴

니다. 우연으로 설명되지 않아서 **차이가 있다**고 말씀드립니다.

p값이 크다는 것은 같다는 뜻이 아닙니다. 우연으로 설명되니 판단을 보류한다는 뜻입니다. 이 구분이 무너지면 앞의 등가성 검정 이야기가 통째로 무너집니다.

Q18. 왜 이 두 제품(pgvector와 오라클)입니까?

저희가 **실제로 운영하던 조합이자, 실제로 검토하던 이전 경로**이기 때문입니다. 딜랩 데모가 pgvector 위에 있었고, 실운영 환경이 오라클이었습니다.

제품 우열을 가리려는 실험이 아닙니다. 저장소와 위치가 함께 바뀌는 구조는 어느 제품 조합의 이전에서나 나타나고, 저희 절차도 제품에 매이지 않습니다.

Q19. 효과 크기란 무엇입니까?

차이의 크기를 표준편차 단위로 나타낸 값입니다.

p값이 "이 차이가 우연인가"를 묻는다면, 효과 크기는 **"그 차이가 실제로 얼마나 큰가"**를 묻습니다. 표본이 아주 많으면 하찮은 차이도 p값이 작게 나올 수 있어서, 두 값을 같이 봐야 합니다.

관례적으로 0.2 근처면 작은 차이, 0.5 근처면 중간, 0.8 이상이면 큰 차이로 봅니다. 저희 결과는 **저장소 요인 0.098로 무시할 수준, 위치 요인 0.473으로 중간**이었습니다.

발표 전 확인 사항

- ☐ 점심시간(12:00-13:30)에 강연장에서 파일 이관 + 발표자 도구 가능 여부 확인 (13:30 이후에는 세션이 연속이라 PC를 만질 틈이 없음)
- ☐ USB에 **rev10 발표용 + PDF 백업**
- ☐ 이 대본 A4 출력 (발표자 도구를 못 쓸 경우의 1차 수단)
- ☐ 14:05까지 앞자리 이동
- ☐ 좌장이 교신저자 성함으로 소개할 수 있음 — 첫 문장에서 본인 이름을 분명히 밝힐 것